

Finding Rules in Data: Association Mining and Market Basket Analysis

BDAT 602 – Data Mining and Applications — Lecture 3

Eric Ocran, PhD

Department of Statistics and Actuarial Science
University of Ghana

Outline

- 1 Motivation and Business Context
- 2 Key Metrics: Support, Confidence, and Lift
- 3 The Apriori Algorithm
- 4 FP-Growth for Big Data
- 5 Sequence Patterns
- 6 Interpreting and Filtering Rules
- 7 Visualising Association Rules
- 8 Interpreting Results for a Business Audience
- 9 Lecture Summary and Looking Ahead

What is Association Rule Mining?

Core Question

“Which items, events, or attributes tend to co-occur frequently in the data?”

Applications beyond retail:

- **Healthcare:**
patients with condition A and B tend to also develop C — informs co-morbidity management.

What is Association Rule Mining?

The classic example — market basket analysis:

- A supermarket analyses millions of transactions
- Discovers: customers who buy *nappies* also tend to buy *beer* on Friday evenings.

Applications beyond retail:

- **Web mining:** users who visit pages X and Y tend to also visit Z — informs recommendation engines.

What is Association Rule Mining?

The classic example — market basket analysis:

- This is non-obvious, previously unknown, and actionable.
- Result: place beer near nappies; bundle them in promotions.

Applications beyond retail:

- **Fraud:** fraudulent claims tend to co-occur with weekend submission and high claim amounts.

What is Association Rule Mining?

The output is a set of rules of the form:

$$\{A, B\} \Rightarrow \{C\}$$

"If a customer has A and B, they also tend to have C."

Applications beyond retail:

- **Genomics:** genes that are expressed together across patients

Association Mining in Our Insurance Dataset

Our business questions:

- 1 Which **add-on riders** tend to be purchased together?
→ Design bundle offers to increase revenue per policyholder
- 2 Which **policyholder characteristics** co-occur with fraud?
→ Flag high-risk claim profiles for manual review

Variables used this week:

- Binary riders:
dental_cover,
vision_cover,
mental_cover,
maternity_cover.

Association Mining in Our Insurance Dataset

Our business questions:

- 1 Which **plan and demographic combinations** co-occur with churn?
→ Design targeted retention interventions

Variables used this week:

- Categorical: plan_tier, region, employment_type, smoker
- Targets: fraud_flag, churned.

Association Mining in Our Insurance Dataset

Policy Records

ID	Items (covers held)
001	Dental, Vision
002	Dental, Mental, Vision
003	Vision, Maternity
004	Dental, Mental
005	Dental, Vision, Mental



Dental $\Rightarrow$ Vision
Support = 0.6,
Lift = 1.8

The Three Rule Metrics: Intuition First

Imagine 1,000 policyholders:

- 600 have dental_cover (D)
- 500 have vision_cover (V)
- 400 have **both** D and V

The rule: $D \Rightarrow V$

1. Support: How common is this combination overall?

$$\text{support}(D \Rightarrow V) = \frac{|D \text{ and } V|}{|\text{all}|} = \frac{400}{1000} = 0.40$$

The Three Rule Metrics: Intuition First

Support = 40%: 40% of all policyholders hold both covers.

2. Confidence: Of those with D, what fraction also has V?

$$\text{confidence}(D \Rightarrow V) = \frac{|D \text{ and } V|}{|D|} = \frac{400}{600} = 0.67$$

Confidence = 67%: among dental holders, 67% also take vision.

The Three Rule Metrics: Intuition First

3. Lift: Is this rule better than random chance?

$$\text{lift}(D \Rightarrow V) = \frac{\text{confidence}(D \Rightarrow V)}{\text{support}(V)} = \frac{0.67}{0.50} = 1.34$$

Lift Interpretation

- = 1 D and V are independent; rule is useless
 - > 1 D and V co-occur more than by chance
 - < 1 D and V are negatively associated
-

The Three Rule Metrics: Intuition First

Which Metric Matters Most?

Support filters rare rules. **Confidence** measures reliability. **Lift** measures genuine association beyond chance. **Always report all three.**

Formal Definitions of Rule Metrics

Let $\mathcal{T}$ be the set of all transactions (records), $|\mathcal{T}| = n$.

Let A and B be itemsets (sets of items/attributes).

The rule is $A \Rightarrow B$ where $A \cap B = \emptyset$.

Support:

$$\text{supp}(A \Rightarrow B) = \frac{|\{t \in \mathcal{T} : A \cup B \subseteq t\}|}{|\mathcal{T}|}$$

Proportion of all transactions containing both A and B .

Formal Definitions of Rule Metrics

Confidence:

$$\text{conf}(A \Rightarrow B) = \frac{\text{supp}(A \cup B)}{\text{supp}(A)}$$

Conditional probability: $P(B \mid A)$.

Lift:

$$\text{lift}(A \Rightarrow B) = \frac{\text{conf}(A \Rightarrow B)}{\text{supp}(B)} = \frac{\text{supp}(A \cup B)}{\text{supp}(A) \cdot \text{supp}(B)}$$

Formal Definitions of Rule Metrics

Additional metrics worth knowing:

Conviction:

$$\text{conv}(A \Rightarrow B) = \frac{1 - \text{supp}(B)}{1 - \text{conf}(A \Rightarrow B)}$$

Measures how often the rule would be incorrect if A and B were independent. Higher = stronger directional dependence.

Leverage:

$$\text{lev}(A \Rightarrow B) = \text{supp}(A \cup B) - \text{supp}(A) \cdot \text{supp}(B)$$

Formal Definitions of Rule Metrics

Difference between observed and expected co-occurrence.
Leverage = 0 means independence.

Lift vs. Confidence

High confidence can occur simply because B is very common.
Lift corrects for this. A rule with confidence = 0.90 but lift = 0.95 is actually *negatively* associated.

Worked Example: Metric Calculations

Setting: 10,000 insurance policyholders.

Item combination	Count
Dental only	2,000
Vision only	1,500
Mental only	1,000
Dental + Vision	2,800
Dental + Mental	1,200
Vision + Mental	800
Dental+Vision+Mental	600
Total with Dental	6,600
Total with Vision	5,700

Worked Example: Metric Calculations

Rule: {Dental} $\Rightarrow$ {Vision}

$$\text{supp} = \frac{2800}{10000} = 0.28$$

$$\text{conf} = \frac{2800}{6600} = 0.424$$

$$\text{lift} = \frac{0.424}{5700/10000} = \frac{0.424}{0.57} = 0.744$$

Worked Example: Metric Calculations

Interpretation

Lift < 1 ! Despite reasonable confidence (42%), dental and vision cover are **negatively** associated — dental holders are actually *less* likely to take vision than the average policyholder. This is a counter-intuitive but important finding.

Setting Thresholds: Minimum Support and Confidence

Why thresholds are necessary: With d binary items, there are 2^d possible itemsets and up to $3^d - 2^{d+1} + 1$ possible rules. For $d = 10$ items (our 4 riders plus 6 other binary variables): this is over 59,000 rules — most are spurious.

Minimum support (minsupp) filters out rules that apply to only a tiny fraction of the data:

- Too high: miss interesting rare rules
- Too low: generate millions of meaningless rules
- **Typical range: 0.01 – 0.10** for insurance data.

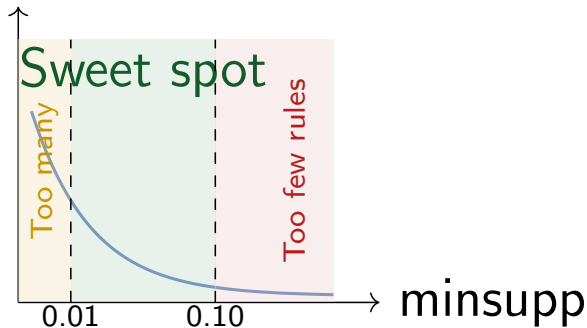
Setting Thresholds: Minimum Support and Confidence

Minimum confidence (minconf) filters out unreliable rules:

- **Typical range: 0.50 – 0.80**
- Always check lift even after filtering by confidence

Setting Thresholds: Minimum Support and Confidence

Rules found



The Apriori Algorithm: Core Idea

The Apriori Principle (Anti-Monotonicity)

If an itemset is **frequent**, then *all* of its subsets must also be frequent.
Equivalently: if an itemset is **infrequent**, then *all* of its supersets must also be infrequent.

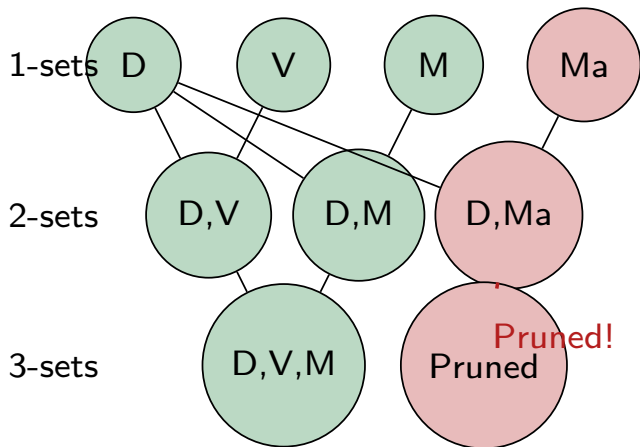
The Apriori Algorithm: Core Idea

Why this is powerful:

- We can **prune** entire branches of the search space
- If {Dental, Vision} is infrequent, we never need to check {Dental, Vision, Mental}
- This reduces the number of candidate itemsets exponentially
- Named “Apriori” because knowledge about subsets is used *apriori* (beforehand)

The Apriori Algorithm: Core Idea

Proposed by: Agrawal & Srikant (1994) — one of the most cited papers in computer science.



The Apriori Algorithm: Step-by-Step

Input: Transaction database $\mathcal{T}$, minsupp s , minconf c **Phase 1: Find all Frequent Itemsets**

- 1 **Pass 1:** Scan $\mathcal{T}$; count each single item; keep items with support $\geq s \rightarrow L_1$ (frequent 1-itemsets)
- 2 **Generate candidates:** combine pairs from $L_1 \rightarrow C_2$
- 3 **Pass 2:** Scan $\mathcal{T}$; count C_2 ; keep those $\geq s \rightarrow L_2$
- 4 **Prune:** any C_2 candidate with an infrequent subset is dropped before scanning
- 5 **Repeat** for $k = 3, 4, \dots$ until no new frequent itemsets found

The Apriori Algorithm: Step-by-Step

Phase 2: Generate Rules from Frequent Itemsets

- ⑥ For each frequent itemset L , generate all non-empty subsets $A \subset L$
- ⑦ For each subset A : output rule $A \Rightarrow (L \setminus A)$ if confidence $\geq c$

Complexity analysis:

- Number of database scans = maximum itemset length
- Each scan is $O(|\mathcal{T}| \times \bar{w})$ where $\bar{w}$ is average transaction width
- Candidate generation is $O(|L_k|^2)$ per level
- **Bottleneck:** repeated full database scans — expensive for large $\mathcal{T}$.

The Apriori Algorithm: Step-by-Step

Limitations of Apriori:

- Multiple passes over the full dataset (one per itemset level)
- Candidate generation produces many itemsets that are later pruned — wasteful
- With 500,000 records and many items, performance degrades
- **Solution:** FP-Growth (Section 4) — only two database scans, no candidate generation

Apriori: A Worked Trace

Five transactions, minsupp = 0.60 (3/5):

TID	Items
T1	Dental, Vision
T2	Dental, Mental, Vision
T3	Vision, Maternity
T4	Dental, Mental
T5	Dental, Vision, Mental

Apriori: A Worked Trace

Pass 1 — item counts:

Item	Count	Frequent?
Dental	4	Yes
Vision	4	Yes
Mental	3	Yes
Maternity	1	No (dropped)

Apriori: A Worked Trace

Pass 2 — 2-itemset counts:

Itemset	Count	Frequent?
{Dental, Vision}	3	Yes
{Dental, Mental}	3	Yes
{Vision, Mental}	2	No

Apriori: A Worked Trace

Pass 3 — 3-itemset:

Itemset	Count	Frequent?
$\{D, V, M\}$	2	No

Frequent itemsets: $\{D\}, \{V\}, \{M\}, \{D, V\}, \{D, M\}$

Generated rules (minconf = 0.60):

$D \Rightarrow V$: conf = $3/4 = 0.75$ ✓

$V \Rightarrow D$: conf = $3/4 = 0.75$ ✓

$D \Rightarrow M$: conf = $3/4 = 0.75$ ✓

R Application: Preparing Transactional Data

```
library(arules)
library(dplyr)
source("simulate_bdat602.R")
health_data <- simulate_bdat602(n = 500000, seed = 602)
# --- Step 1: Select and discretise variables ---
# For association mining, ALL variables must be
#   categorical/binary
# Continuous variables must be binned first
trans_df <- health_data |> mutate(
# Binary riders (already 0/1 --- convert to logical
#   labels)
dental      = ifelse(dental_cover == 1, "Dental",    NA),
vision      = ifelse(vision_cover == 1, "Vision",    NA),
mental      = ifelse(mental_cover == 1, "Mental",    NA),
```

R Application: Preparing Transactional Data

```
maternity = ifelse(maternity_cover== 1, "Maternity", NA),  
# Discretise continuous variables into meaningful bins  
age_grp = cut(age, breaks = c(17, 30, 45, 60, 85),  
labels = c("Young", "MidAge", "Senior", "Elderly")),  
bmi_grp= cut(bmi, breaks = c(0, 18.5, 25, 30, 100),  
labels = c("Underweight", "Normal", "Overweight", "Obese")),  
# Keep categorical variables as-is  
fraud_lbl = ifelse(fraud_flag == 1, "Fraud", "NoFraud"),  
churn_lbl = ifelse(churned== 1, "Churned", "Retained"),  
smoker_lbl= ifelse(smoker == 1, "Smoker", "NonSmoker") )  
|>  
select(dental, vision, mental, maternity, plan_tier,  
       region, age_grp, bmi_grp, employment_type, smoker_lbl,  
       fraud_lbl, churn_lbl)
```

R Application: Creating the Transactions Object

```
library(arules)
# --- Step 2: Convert to arules transactions format ---
# Each row = one policyholder's "basket" of attributes
# Method A: from a data frame of factors/characters
trans_obj <- as(trans_df, "transactions")
# Inspect the transaction object
summary(trans_obj)
# Reports: number of transactions, items, density,
# most frequent items, transaction size distribution
# --- Item frequency plot: what are the most common items
? ---
```

R Application: Creating the Transactions Object

```
itemFrequencyPlot(trans_obj, topN = 20, type = "absolute",  
main = "Top 20 Most Frequent Items in Insurance  
Transactions",  
col = "steelblue", las = 2, cex.names = 0.75)  
# --- Inspect a few transactions ---  
inspect(trans_obj[1:5])
```

What a Transaction Looks Like

Each row in `trans_obj` is a set of items (attribute values) present for that policyholder. For example: {Dental, Vision, Gold, MidAge, Normal, NonSmoker, Retained}.

R Application: Running Apriori

```
library(arules)
# --- Run Apriori algorithm ---
rules_apriori <- apriori(trans_obj,
  parameter = list(supp = 0.02, # at least 2% of
    policyholders
  conf = 0.60, # at least 60% confidence
  minlen = 2, # minimum rule length (LHS + RHS items)
  maxlen = 5 # maximum rule length)) # How many rules were
  found?
summary(rules_apriori) # Inspect top 15 rules sorted by
  lift
inspect(sort(rules_apriori, by="lift", decreasing=TRUE)
  [1:15])
```

R Application: Running Apriori

```
# --- Focus on rules with fraud on the RHS ---  
fraud_rules<- subset(rules_apriori, subset = rhs %in% "  
  fraud_lbl=Fraud" & lift > 1.5)  
inspect(sort(fraud_rules, by = "lift"))
```

Why Apriori Struggles at Scale

Apriori's scalability problem:

- Requires **one full database scan per itemset level**
- With 500,000 records and low minsupp, there may be many levels
- **Candidate generation:** even after pruning, millions of candidates may be generated at each level
- Candidates are stored in memory and checked against the database — this is the memory bottleneck

Why Apriori Struggles at Scale

Example with our data:

- 12 items (4 riders + plan tier + region + age group + ...)
- minsupp = 0.01 (1%)
- Apriori may generate thousands of candidates at level 3 and beyond
- Each requires a scan of 500,000 records

FP-Growth (Frequent Pattern Growth) solution:

- 1 Only two database scans — ever
- 2 Compresses the database into a compact **FP-Tree** (Frequent Pattern Tree) data structure
- 3 Mines the FP-Tree **without generating candidates**
- 4 Uses a divide-and-conquer strategy: mines one item at a time via conditional pattern bases

Performance

FP-Growth is typically **10–100× faster** than Apriori on large datasets. It is the algorithm of choice for big data association mining.

The FP-Tree Data Structure

Building the FP-Tree (2 database scans):

1. **Scan 1:** Count item frequencies; remove infrequent items; sort remaining items by frequency (descending)
2. **Scan 2:** For each transaction, insert items (in frequency order) into the tree:
 - If a path already exists: increment node counts
 - If not: add new branches
 - A **header table** links all nodes for each item across the tree

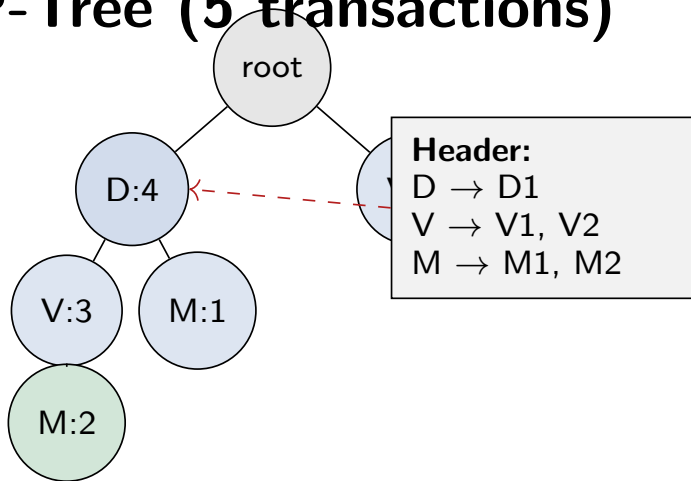
Building the FP-Tree (2 database scans):

Mining the FP-Tree:

1. For each item i (least frequent first): extract the **conditional pattern base** (all paths ending at i) and build a smaller conditional FP-Tree
2. Recursively mine each conditional FP-Tree

The FP-Tree Data Structure

FP-Tree (5 transactions)



R Application: FP-Growth in R

```
library(arules)
# --- FP-Growth in arules ---
# The arules package uses FP-Growth internally when
# eclat() is called, and also supports it via apriori()
# with optimised implementations for large data
# --- Method 1: ECLAT (Equivalence Class Transformation)
# Depth-first search using vertical data format;
# similar efficiency to FP-Growth for itemset mining
rules_eclat <- eclat(trans_obj, parameter = list(supp=
  0.02,minlen = 2, maxlen = 5))
# Convert frequent itemsets to rules
rules_from_eclat <- ruleInduction(rules_eclat,trans_obj,
  confidence = 0.60)
summary(rules_from_eclat)
inspect(sort(rules_from_eclat, by = "lift")[1:10])
```

Apriori vs. ECLAT vs. FP-Growth in R

The `arules` package implements optimised versions of all three. For datasets with $n > 100,000$ records, always prefer `eclat()` or tune `apriori()` with higher `minsupp` to reduce candidate generation.

Scaling Association Mining with Spark

```
library(sparklyr)
library(dplyr)
sc <- spark_connect(master = "local[*]", version = "3.4.1")
# Copy transactions to Spark
health_tbl <- copy_to(sc, health_data,
name = "health_ins", overwrite = TRUE)
# --- FP-Growth in Spark via sparklyr ---
# ml_fpgrowth() implements the distributed FP-Growth
  algorithm
# Items must be in a list-column format
# Prepare: create item list column
```


Scaling Association Mining with Spark

```
health_items <- health_tbl |>
mutate(items = array(
  ifelse(dental_cover == 1, "Dental", NULL),
  ifelse(vision_cover == 1, "Vision", NULL),
  ifelse(mental_cover == 1, "Mental", NULL),
  ifelse(maternity_cover == 1, "Maternity", NULL),
  plan_tier, region)) |> select(record_id, items)
# Run FP-Growth in Spark
fp_model <- health_items |>
ml_fpgrowth(items_col = "items",
  min_support = 0.02, min_confidence = 0.60)
# Extract association rules
spark_rules <- ml_association_rules(fp_model)
spark_rules |> collect() |> arrange(desc(lift)) |>
head(15)
```

Beyond Co-occurrence: Sequence Patterns

Standard association rules capture co-occurrence but **ignore order**. Sometimes the *sequence* of events matters:

- A policyholder files a small dental claim, then a large hospital claim within 6 months — is this a fraud pattern?
- Customers who call support twice tend to churn within 90 days
- Patients who are prescribed drug A tend to be prescribed drug B within 30 days

Beyond Co-occurrence: Sequence Patterns

Sequential Pattern Mining

Given a database of ordered sequences of events, find all **frequent subsequences** — ordered patterns that appear in at least minsupp fraction of sequences.

Notation: $\langle A \rangle \rightarrow \langle B \rangle \rightarrow \langle C \rangle$ means A occurs, then B, then C (in that order).

Beyond Co-occurrence: Sequence Patterns

Example sequences from our data:

Policy ID	Event Sequence
P001	Visit → Prescription → Claim
P002	Claim → SupportCall → Churn
P003	Visit → Claim → Claim
P004	SupportCall → SupportCall → Churn
P005	Prescription → Claim

Beyond Co-occurrence: Sequence Patterns

Frequent subsequence (minsupp=0.4):

$\langle \text{Claim} \rangle \rightarrow \langle \text{Churn} \rangle$: appears in P002 and P004 if we generalise — support = 0.40.

Key algorithm: GSP (Generalised Sequential Patterns) and PrefixSpan

R Application: Sequence Mining with arulesSequences

```
library(arulesSequences)
library(dplyr)
# --- Build a sequence database from claims history ---
# Each policyholder has a sequence of events over time
# Construct event sequences: one row per event per
  policyholder
event_seq <- health_data |> filter(num_claims > 0) |>
select(policyholder_id, policy_start_date,
last_claim_date, support_calls, churned) |>
mutate(event1 = "Claim",
event2 = ifelse(support_calls > 0, "SupportCall", NA),
event3 = ifelse(churned == 1, "Churn", NA)) |>
tidyr::pivot_longer(cols = starts_with("event"),
```

R Application: Sequence Mining with arulesSequences

```
values_to = "event") |> filter(!is.na(event)) |> arrange(
  policyholder_id)
# Convert to transaction sequences format
# (arulesSequences requires a specific format)
# See package vignette for full specification:
# vignette("arulesSequences")
# Mine frequent sequences
seq_rules <- cspade(
  as(event_seq, "transactions"), # after proper formatting
  parameter = list(support = 0.02),
  control    = list(verbose = TRUE))
inspect(seq_rules)
```

The Problem of Too Many Rules

A common outcome after running Apriori:

- With lenient thresholds: **thousands** of rules are generated
- Many are **redundant**: if $\{A\} \Rightarrow \{C\}$ holds, then $\{A, B\} \Rightarrow \{C\}$ is less general but may also pass thresholds
- Many are **trivial**: rules that simply reflect the base rate of a very common item (high support + low lift)
- Some are **spurious**: pass thresholds by chance in a large dataset

The Problem of Too Many Rules

Strategies for managing the rule set:

- 1 Raise minsupp and minconf thresholds
- 2 Filter by **lift** > 1 (removes negatively associated rules)
- 3 Remove **redundant rules**: a rule is redundant if a more general rule exists with equal or higher confidence
- 4 Filter by **specific RHS items** of business interest
- 5 Sort by **conviction** or **leverage** for alternative ordering

The Problem of Too Many Rules

Redundancy example:

Rule 1: $\{D\} \Rightarrow \{M\}$, $\text{conf} = 0.72$, $\text{lift} = 1.8$

Rule 2: $\{D, V\} \Rightarrow \{M\}$, $\text{conf} = 0.72$, $\text{lift} = 1.8$

Rule 2 is **redundant**: adding V to the LHS does not improve confidence or lift. Rule 1 is more general and equally informative — prefer Rule 1.

Business Relevance Filter

Always filter rules to those where the **RHS** (consequent) is the item of business interest. For fraud detection: keep only rules where $\text{RHS} = \{\text{Fraud}\}$.

R Application: Filtering and Inspecting Rules

```
library(arules)
library(dplyr)
# --- Filter rules by multiple criteria ---
# 1. Keep only rules with lift > 1.2 (positively
  associated)
strong_rules <- subset(rules_apriori, subset = lift > 1.2)
# 2. Focus on churn-related rules (churn on the RHS)
churn_rules <- subset(rules_apriori,
  subset = rhs %in% "churn_lbl=Churned" & lift > 1.3 &
    confidence > 0.65)
inspect(sort(churn_rules, by = "lift"))
```

R Application: Filtering and Inspecting Rules

```
# 3. Remove redundant rules
non_redundant <- rules_apriori[!is.redundant(rules_
  apriori)]
cat("Rules before:", length(rules_apriori), "\n")
cat("Rules after removing redundant:", length(non_
  redundant), "\n")

# 4. Convert rules to a data frame for further analysis
rules_df <- as(non_redundant, "data.frame")
head(rules_df |> arrange(desc(lift)), 10)

# 5. Find rules containing a specific item anywhere
dental_rules <- subset(rules_apriori,
  items %in% "dental=Dental")
inspect(sort(dental_rules, by = "confidence")[1:10])
```

Visualising Association Rules: Overview

Why visualise?

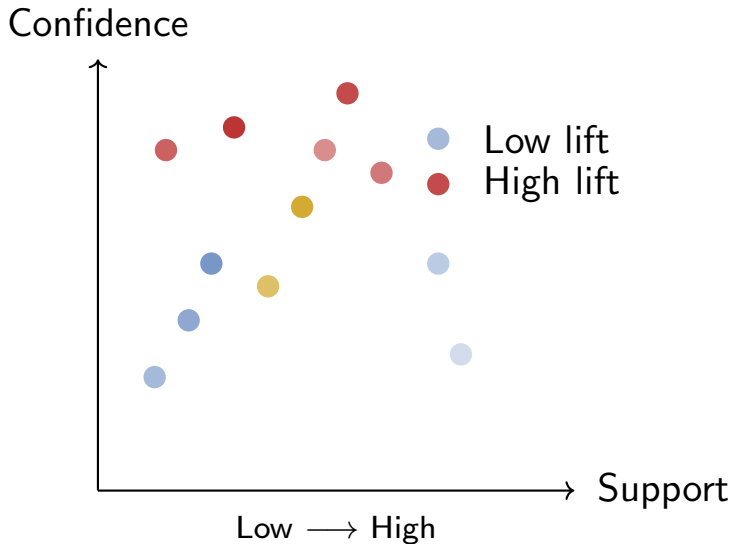
- Even after filtering, we may have hundreds of rules
- Tables of rules are hard to scan for patterns
- Visualisation reveals **clusters of related rules** and **outliers** (unusually high lift)
- Essential for communicating findings to non-technical stakeholders

Visualising Association Rules: Overview

The `arulesViz` package provides:

- **Scatter plot:** support vs. confidence, coloured by lift
- **Two-key plot:** a variant where size = support, colour = confidence
- **Matrix plot:** items on axes, colour = lift
- **Graph plot:** items as nodes, rules as edges, thickness = lift
- **Parallel coordinates:** each rule as a line across item axes
- **Grouped matrix:** hierarchical grouping of rules

Scatter Plot: Support vs Confidence



R Application: Visualising Rules with arulesViz

```
library(arulesViz)
# --- 1. Scatter plot: support vs confidence, coloured by
# lift ---
plot(rules_apriori, measure = c("support", "confidence"),
     shading="lift",
     main = "Association Rules: Support vs Confidence (shade =
           Lift)",
     col = colorRampPalette(c("steelblue", "gold", "red"))(100))
#--2. Graph plot: items as nodes, rules as directed edges
# Use only top 30 rules by lift for readability
top30 <- sort(rules_apriori, by = "lift")[1:30]
plot(top30, method = "graph", engine = "htmlwidget")
# interactive; opens in browser
```


R Application: Visualising Rules with arulesViz

```
# --- 3. Grouped matrix plot ---
plot(rules_apriori,
method    = "grouped",
control   = list(k = 10),
main      = "Grouped Matrix of Association Rules")
# --- 4. Parallel coordinates plot ---
plot(top30, method = "paracoord", control = list(reorder =
  TRUE))
```

R Application: Interpreting the Graph Plot

```
library(arulesViz)
library(igraph)
# --- Detailed graph plot with custom aesthetics ---
plot(top30, method = "graph", engine = "igraph",
     control = list(layout = igraph::layout_with_fr,
                    nodeCol = grey.colors(10), edgeCol = "steelblue",
                    arrowSize = 0.5,
                    precision = 3),
     main = "Top 30 Rules by Lift: Graph Visualisation")
# Reading the graph:
# - Circles = ITEMS (attributes)
# - Squares = RULES
# - Arrow from item circle to rule square = LHS item
# - Arrow from rule square to item circle = RHS item
# - Larger square = higher
```

R Application: Interpreting the Graph Plot

```
# - Larger square = higher support
# - Darker colour = higher lift
# --- Which items appear most in high-lift rules? ---
rules_df <- as(top30, "data.frame")
cat("Top consequents (RHS) in high-lift rules:\n")
table(rules_df$rules) |> sort(decreasing = TRUE) |> head
(10)
```

From Rules to Recommendations

Suppose Apriori finds this rule:

Rule: $\{\text{Gold, Dental, NonSmoker}\} \Rightarrow \{\text{Vision}\}$
Support = 0.08, Confidence = 0.74, Lift = 1.92

What does this mean in plain language?

- 8% of all policyholders hold a Gold plan, dental cover, and are non-smokers *and* have vision cover.
- Among Gold/Dental/NonSmoker policyholders, **74%** also have vision cover.

From Rules to Recommendations

- Vision cover is **1.92 times more likely** among this group than among the general portfolio.

Business recommendation:

- Target Gold plan, non-smoking, dental holders with **vision cover bundle promotions**.
- The conversion rate is expected to be nearly double the portfolio average.

From Rules to Recommendations

A fraud-related rule:

Rule: $\{\text{Unemployed, Young, WeekendClaim}\} \Rightarrow \{\text{Fraud}\}$

Support = 0.012, Confidence = 0.52, Lift = 4.1

Interpretation:

- Rare but strong: only 1.2% of records match this profile.
- Among these policyholders, 52% are flagged as fraudulent.
- This profile is **4.1**× **more likely** to commit fraud than average.

From Rules to Recommendations

Recommendation:

- Route all claims matching this profile to the **manual review queue**.
- The rule covers only 1.2% of claims — manageable workload.

Common Pitfalls in Association Mining

1. Confusing correlation with causation

- A rule $A \Rightarrow B$ does not mean A *causes* B
- The nappies-beer rule: being a new parent (unobserved) causes both purchases
- Always seek a *plausible mechanism* before acting on a rule

Common Pitfalls in Association Mining

2. Ignoring lift and using only confidence

- High confidence can occur when B is simply very common.
- A rule with confidence = 0.80 but lift = 0.85 is *negatively* associated.
- Always report lift alongside confidence.

3. Mining rules without a business question

- “Let us mine everything and see” produces an uninterpretable mass of rules.
- Always define the RHS item of interest before mining.

4. The Simpson's Paradox problem

- A rule may hold overall but reverse within subgroups
- Always stratify and check rules within meaningful segments (by region, plan tier)

5. Spurious rules in large datasets

- With $n = 500,000$ records, even rare combinations appear frequently purely by chance.
- Apply a **statistical significance filter**: use the hypergeometric test or Fisher's exact test to check whether the co-occurrence is beyond chance.
- arules supports this via `interestMeasure(rules, "fishersExactTest")`

6. Not removing redundant rules

- A redundant rule adds no new information but inflates the rule set
- Always run `is.redundant()` before presenting results

R Application: Statistical Significance of Rules

```
library(arules)
# --- Compute additional interest measures ---
measures <- interestMeasure(rules_apriori,
measure = c("support", "confidence", "lift","conviction",
            "leverage",
"fishersExactTest"),transactions = trans_obj)
# Attach measures to rules data frame
rules_df <- as(rules_apriori, "data.frame")
rules_df <- cbind(rules_df, measures)
# Filter: lift > 1.5 AND Fisher p-value < 0.01 (
            statistically significant)
significant_rules <- rules_df |> filter(lift > 1.5,
fishersExactTest < 0.01) |> arrange(desc(lift))
```

R Application: Statistical Significance of Rules

```
nrow(significant_rules)
# Display top rules
significant_rules |>
select(rules, support, confidence, lift,
conviction, fishersExactTest) |>
head(15)
```

Why Statistical Testing Matters at Scale

With 500,000 records, even spurious co-occurrences achieve high support by chance. Fisher's exact test confirms that the association is **genuinely non-random**, not just frequent.

What We Covered Today

Conceptual foundations:

- ✓ What association rule mining is and where it applies beyond retail
- ✓ The three core metrics: support, confidence, and lift — with intuition and formal definitions
- ✓ Additional metrics: conviction and leverage

Practical (R):

- ✓ Discretising continuous variables for transactional format
- ✓ Creating transactions objects with `arules`
- ✓ Running Apriori with `apriori()`

What We Covered Today

Conceptual foundations:

- ✓ Choosing minimum support and confidence thresholds
- ✓ The Apriori principle (anti-monotonicity) and its step-by-step trace
- ✓ Why Apriori struggles at scale and how FP-Growth solves this

Practical (R):

- ✓ Distributed FP-Growth with `sparklyr::ml_fpgrowth()`
- ✓ Filtering rules with `subset()`, `is.redundant()`
- ✓ Statistical significance with Fisher's exact test

What We Covered Today

Conceptual foundations:

- ✓ The FP-Tree data structure and conditional pattern bases
- ✓ Sequence pattern mining for ordered events
- ✓ Filtering redundant and spurious rules
- ✓ Common pitfalls: causation, Simpson's Paradox, lift neglect

Practical (R):

- ✓ Visualising rules with `arulesViz`: scatter, graph, grouped matrix, parallel coordinates
- ✓ Translating rules into plain-language business recommendations

Next Week: Grouping Similar Things — Cluster Analysis

Having discovered rules about *which attributes co-occur*, we now ask: *which policyholders resemble each other enough to be grouped together?* Clustering finds natural segments in the data without any predefined labels.

Looking Ahead: Lecture 4

Key topics in Lecture 4:

- k -means clustering: algorithm and the elbow method for choosing k
- Hierarchical clustering and dendrogram reading
- DBSCAN for irregular shapes and noise
- Silhouette score: evaluating cluster quality

Dataset continuity:

- Target: segment the 500,000 policyholders into meaningful risk profiles
- Features: age, bmi, income, num_chronic_conditions, num_claims, premium, support_calls, customer_rating

Looking Ahead: Lecture 4

Key topics in Lecture 4:

- Scaling clustering to 500,000 rows with `sparklyr::ml_kmeans()`

Dataset continuity:

- The PCA scores computed in Lecture 2 will be used for 2D cluster visualisation

R tools:

- `cluster`, `factoextra`, `dbscan`, `sparklyr`

Key Takeaways

Three Rules of Association Mining

- 1 **Never report confidence without lift.** A rule with confidence = 0.85 and lift = 0.90 describes a *negative* association. Lift tells you whether the rule beats the baseline.
- 2 **Define your business question before running the algorithm.** Specify the RHS item of interest before calling `apriori()`. Mining without a question produces noise, not insight.

Key Takeaways

Three Rules of Association Mining

- 1 **Remove redundant rules and test for statistical significance.** At $n = 500,000$, spurious rules pass support thresholds by chance. Always run `is.redundant()` and Fisher's exact test before presenting results.

Algorithm	Strength	Use When
Apriori	Simple; interpretable	Small/medium datasets; teaching
ECLAT	Faster than Apriori	Medium datasets; itemset-first mining
FP-Growth	Fastest; no candidate generation	Large datasets ($n > 100,000$)
GSP/PrefixSpan	Ordered sequences	Event logs; time-stamped data

Self-Study and Further Reading

Core reading for Lecture 3:

- Han, Kamber & Pei (2011), *Data Mining*:
Ch. 6: Mining Frequent Patterns, Associations, and Correlations
Ch. 7: Advanced Pattern Mining (FP-Growth, sequences)
- Tan, Steinbach & Kumar (2019):
Ch. 5: Association Analysis: Basic Concepts and Algorithms
Ch. 6: Association Analysis: Advanced Concepts
- Leskovec et al. (2020), *MMDS*:
Ch. 6: Frequent Itemsets (free at mmds.org)

Self-Study and Further Reading

R resources:

- arules vignette:
`cran.r-project.org/package=arules`
- arulesViz vignette:
`cran.r-project.org/package=arulesViz`
- Hahsler et al. (2011), *Introduction to arules*,
R Journal

Self-Study and Further Reading

Practice exercises:

- Mine rules from the insurance dataset with $\text{minsupp} = 0.05$, $\text{minconf} = 0.70$; how many survive after removing redundant rules and filtering $\text{lift} > 1.5$?
- Produce a graph plot of the top 20 rules by lift and identify the most central item node
- Compare Apriori and ECLAT runtime on the full 500,000-row dataset using `system.time()`

Questions?

Next: Lecture 4 — Grouping Similar Things:
Cluster Analysis

BDAT 602 · Data Mining and Applications · University of Ghana